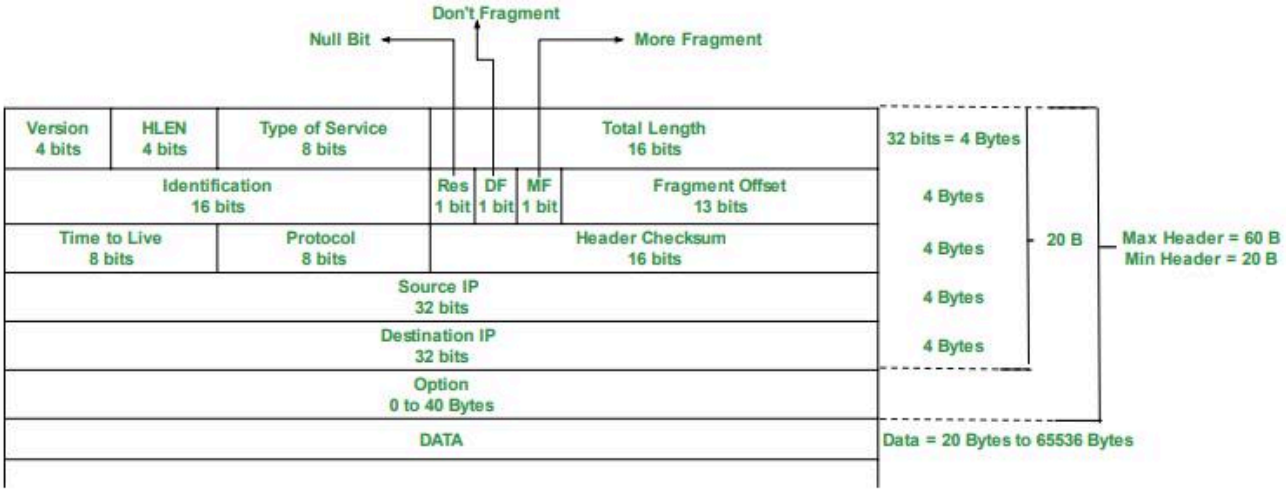
Lab 2 - ICMP Pinger Report

Approach Overview

First, I needed to find out how to fetch the ICMP header from the IP packet and that requires knowing exactly how long the base IPv4 packet header is and where the ICMP header exists in the IP packet.

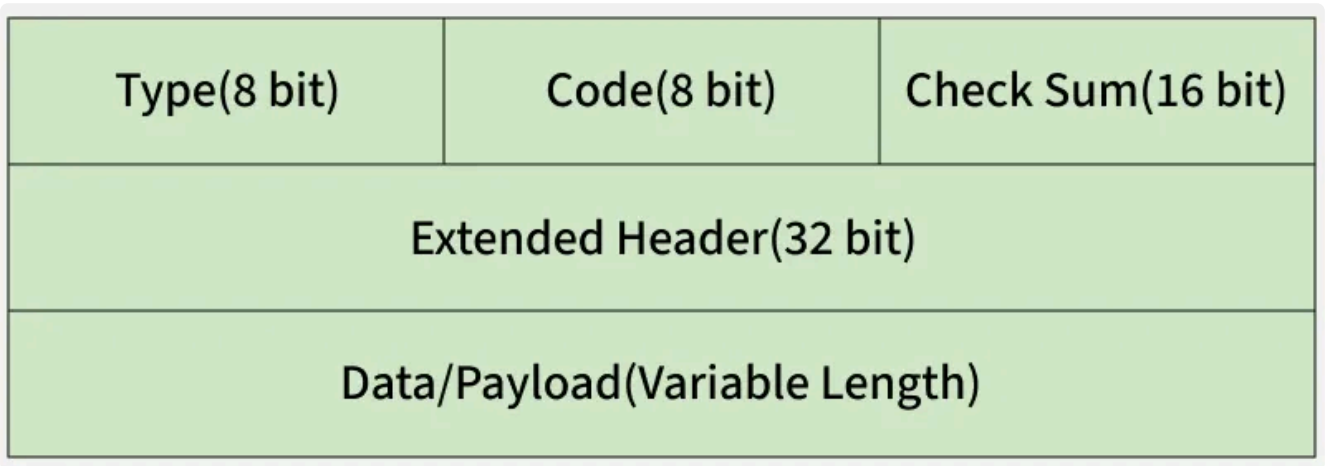
After discovering that information I would then need to analyze the sendOnePing() function format to discover the syntax needed to reverse the process and then implement the receiving equivalent for receiveOnePing().

IP Packet Format

> Due to the presence of options, the size of the datagram header can be of variable length (20 bytes to 60 bytes).
>
> 
>
>
> geeksforgeeks [1]

Using the reference from above, I will begin by assuming that the size of our IPv4 header is 20 bytes long. With this assumption, the 161st bit should be the beginning of our ICMP header.

ICMP Packet Format

> ICMP header comes after IPv4 and IPv6 packet header. In the ICMP packet format, the first 32 bits of the packet contain three fields:
>
> 
>

- **Type (8-bit):** The initial 8-bit of the packet is for message type, it provides a brief description of the message so that receiving network would know what kind of message it is receiving and how to respond to it. Some common message types are as follows:

```
Type 0 - Echo reply
Type 3 - Destination unreachable
Type 5 - Redirect Message
Type 8 - Echo Request
Type 11 - Time Exceeded
Type 12 - Parameter problem
```

- **Code (8-bit):** Code is the next 8 bits of the ICMP packet format, this field carries some additional information about the error message and type.
- **Checksum (16-bit):** Last 16 bits are for the checksum field in the ICMP packet header. The checksum is used to check the number of bits of the complete message and enable the ICMP tool to ensure that complete data is delivered.
- The next 32 bits of the ICMP Header are Extended Header which has the work of pointing out the problem in IP Message. Byte locations are identified by the pointer which causes the problem message and receiving device looks here for pointing to the problem.
- The last part of the ICMP packet is Data or Payload of variable length. The bytes included in IPv4 are 576 bytes and in IPv6, 1280 bytes.
  geeksforgeeks [2]

With the reference above I can tell that the entire ICMP header will consume 8 total bytes for `Type`, `Code`, `Checksum`, and `Extended Header` fields. The `Extended Header` field is broken up into two, two byte sections `ID` and `Sequence`.

Checking `sendOnePing()` Func Format

`struct.pack()`

> The format string in struct.pack() defines how data should be stored in bytes. Some common format codes are:

Format Code	Data Type	Size (bytes)
b	Signed char	1
B	Unsigned char	1
h	Signed short	2
H	Unsigned short	2
i	Signed int	4
I	Unsigned int	4
f	Float	4
d	Double	8

> geeksforgeeks [3]

So what did this code mean?

```
header = struct.pack("bbHHh", ICMP_ECHO_REQUEST, 0, myChecksum, ID, 1)
```

After reviewing the common format codes used, I can see that:

- `"bbHHh"` maps nicely to our ICMP header fields
  - `ICMP_ECHO_REQUEST` is the one byte `type` field and hardcoded to `8`
  - The one byte `code` field is hardcoded to `0`
  - `mychecksum` maps to the two byte check sum field
  - The four byte extended header field is filled in by the two byte current process ID as an unsigned short and then the remaining two bytes as a signed short representation of `1`

`time.time()`

Now, the identification of the data field of the packet is necessary.

> The Python time **time()** method returns the current UTC time. This return value is obtained as a floating point number expressed in seconds since the epoch.
> tutorials point [4]

Although the `time()` method returns a float, its packed as a double so when its unpacked it will need to be unpacked as eight bytes, not four.

Reversing Logic for `receiveOnePing()`

> struct.unpack(*format*, *buffer*)
> Unpack from the buffer *buffer* (presumably packed by `pack(format, ...)`) according to the format string *format*. The result is a tuple even if it contains exactly one item. The buffer's size in bytes must match the size required by the format
> python docs [5]

Firstly, our buffer should be the 21st byte of our `recPacket` or index 20. Then the ICMP header should take up the next 7 indices for a total of 8 bytes:

```
icmp_header = recPacket[20:28] # slices 8 byte ICMP header from recPacket
(indices 20-27)
```

So, `struct.unpack(format, buffer)` should simply return a tuple of five items just as they were packed:

```
icmp_type, icmp_code, icmp_checksum, icmp_id, icmp_sequence =
struct.unpack("bbHHh", icmp_header)
```

Then we can unpack the time value returned from `time()` method in our next 8 bytes after the header:

```
icmp_time_data = recPacket[28:36]
icmp_time_sent = struct.unpack('d', icmp_time_data)[0]
```

Finally, we can check if:

1. the ID passed to the `receiveOnePing()` func is the same as the `icmp_id` we received
2. that the `icmp_type` is equal to zero for a echo reply
   Then, if those are true, we can find the ICMP ping RTT and return that to the caller:

```
if ID == icmp_id and icmp_type == 0:
    return timeReceived - icmp_time_sent
```

Testing

> **Note**
> My code initially wouldn't work, but after asking Claude to help me debug the code I changed the one line in the screenshot below because the original code snippet

provided was corrupting the checksum because it was made for Python 2 code.

```
# Calculate the checksum on the data and the dummy header.
myChecksum = checksum(str(header + data))
myChecksum = checksum((header + data).decode('latin-1'))
```

> This is the core fix. str(bytes_object) in Python 3 gives you a string representation like "b'\x08...'", which completely corrupts the checksum. .decode('latin-1') gives you a 1-to-1 character mapping of each byte, which is what the ord() calls in the checksum function expect.

I used this tool https://www.meter.net/tools/world-ping-test/ for finding 4 locations around the world to ping as instructed.

Local Host

Testing ping with local host:

```
ping("127.0.0.1")
```

Output:

```
genki@Genki:~/Projects/WebServer/Project2$ sudo python3 ICMP_Pinger.py
Pinging 127.0.0.1 using Python:

8.487701416015625e-05
0.00010442733764648438
6.318092346191406e-05
0.00011181831359863281
8.106231689453125e-05
```

Google

Testing ping with google.com:

```
ping("google.com")
```

Output:

```
genki@Genki:~/Projects/WebServer/Project2$ sudo python3 ICMP_Pinger.py
Pinging 142.251.35.142 using Python:

0.19934701919555664
0.005083322525024414
0.19347190856933594
0.005711078643798828
0.005250692367553711
```

Delhi, India VULTR.net

Testing ping with VULTR.net

```
ping("inde-vltr.metercdn.net")
```

Output:

```
genki@Genki:~/Projects/WebServer/Project2$ sudo python3 ICMP_Pinger.py
Pinging 139.84.167.59 using Python:

0.27300357818603516
0.27364540100097656
0.27388429641723633
0.2741425037384033
```

Sydney, Australia VULTR.net

Testing ping with VULTR.net:

```
ping("au-ml-vltr.metercdn.net")
```

Output:

```
genki@Genki:~/Projects/WebServer/Project2$ sudo python3 ICMP_Pinger.py
Pinging 67.219.102.28 using Python:

0.23651385307312012
0.237288236618042
0.2362840175628662
0.23610639572143555
0.24402713775634766
```

Sao Paulo, Brazil VULTR.net

Testing ping VULTR.net:

```
ping("au-ml-vltr.metercdn.net")
```

Output:

```
genki@Genki:~/Projects/WebServer/Project2$ sudo python3 ICMP_Pinger.py
Pinging 216.238.111.192 using Python:

0.15188932418823242
0.1531221866607666
0.1555490493774414
0.16241121292114258
```

Paris, France VULTR.net

Testing ping with VULTR.net

```
ping("fr-prvltr.metercdn.net")
```

Output:

```
genki@Genki:~/Projects/WebServer/Project2$ sudo python3 ICMP_Pinger.py
Pinging 104.238.190.162 using Python:

0.11481213569641113
0.11502933502197266
0.11631226539611816
0.11514544486999512
```

References

1. https://www.geeksforgeeks.org/computer-networks/introduction-and-ipv4-datagram-header/
2. https://www.geeksforgeeks.org/computer-networks/internet-control-message-protocol-icmp/
3. https://www.geeksforgeeks.org/python/struct-pack-in-python/
4. https://www.tutorialspoint.com/python/time_time.htm
5. https://docs.python.org/3/library/struct.html